

Foundations of Data Science

Zou Weiyuan

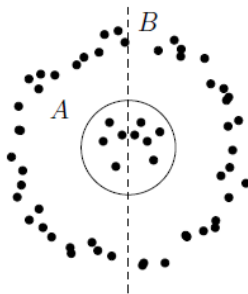
May 9, 2019

Overview

- 1 7.7 High-Density Clusters
- 2 7.8 Kernel Methods
- 3 7.9 Recursive Clustering based on Spared Cuts
- 4 7.10 Dense Submatrices and Communities
- 5 7.11 Community Finding and Graph Partition
- 6 Summary

High-Density Clusters

Consider the clusters that are center-based: assume that cluster consist of high-density regions, separated by low-density moats such as:



Density cluster.png

Def of Single Linkage

Single Linkage: Algorithm begins with each point in its own clusters and then repeatedly merges the two "closest" clusters into one.

Remark

The distance between two clusters is defined as the minimum distance between points in each clusters. That is,

$$d_{min}(C, C') = \min_{x \in C, y \in C'} d(x, y).$$

Kruskal's minimum-Spanning-Tree algorithm

Theorem (Kruskal's minimum-Spanning-Tree algorithm)

Suppose the desired clustering $C_1^, \dots, C_k^*$ satisfies the property that there exists some distance σ such that*

- 1 any two data points in different clusters have distance at least σ , and*
- 2 for any cluster C_i^* and any partition of C_i^* into two non-empty sets A and $C_i^* \setminus A$, there exist points on each side of the partition of distance less than σ .*

Proof of Kruskal's minimum-Spanning-Tree algorithm

Proof:

Consider Running the algorithm until all pairs of clusters C and C' have $d_{\min}(C, C') \geq \sigma$. At that point, by(2), each target cluster C_i^* will be fully contained within some cluster of the single-linkage algorithm.

On the other hand, by (1) and by induction, each cluster C of the single-linkage algorithm will be fully contained within some C_i^* of the target clustering, since any merger of subsets of distinct target clusters would require $d_{\min} \geq \sigma$. Therefore, the single-linkage clusters are indeed the target cluster.

Robust Linkage

Aim: The single-linkage algorithm is fairly brittle. A few points bridging the gap between two different clusters can cause it to do the wrong thing. As a result, there has been significant work developing **more robust versions of the algorithm**.

Wishart's algorithm

A ball of radius r is created for each point with the point as center; The radius is gradually increased starting from $r = 0$.

The algorithm has a parameter t , when a ball has t or more points the center of point becomes active. When the two balls with active centers intersect the two center points are connected by an edge. The parameter t prevents a thin string of points between two clusters from causing a spurious merger.

Remark:

$t=1$, Wishart's algorithm is same as single linkage.

Assumption of Modified algorithm

- 1 for some distance σ , the σ -interior of each target cluster C_i^* has density at least some quantity λ .
- 2 the region between target clusters has density less than $\lambda(1 - \epsilon)$ for some $\epsilon > 0$,
- 3 the clusters should be separated by distance greater than 2σ , and
- 4 the σ -interior of the clusters contains most of their probability mass.

Kernel Methods

The clustering methods discussed so far work well only when the data satisfy certain conditions.

For example, in any distance-based measure like k-means or k-center, once the cluster centers are fixed, the Voronoi diagram of the cluster centers determines which cluster each data point belongs to.

The chapter on learning had many examples that were not linearly separable in the original space, but were linearly separable when mapped to a higher dimensional space using a nonlinear function called a kernel. An analogous technique can be used in the case of clustering, but with two differences.

Two differences

- 1. There may be any number k of clusters, whereas in learning, there were just two classes, the positive and negative examples.
- 2. There is unlabelled data, i.e., we are not given which cluster each data point belongs to, whereas in the case of learning each data point was labeled. The clustering situation is sometimes called unsupervised whereas the labeled learning situation is called supervised, the reason being, one imagines a supervisor, human judgement, supplying the labels.

Kernel Methods

Recall that we only know dot products, not distances in the higher dimensional space, but we can use the relation $|z_1 - z_2|^2 = z_1 \cdot z_1 - 2z_1 \cdot z_2 + z_2 \cdot z_2$ to go from dot products to distances. One popular kernel function to use is the Gaussian kernel. Specially, we define the affinity points x and y by

$$K(x, y) = e^{-\frac{1}{2\sigma^2} \|x-y\|^2}.$$

Definition (*Recursive Clustering*)

Select an approximate value for ϵ . If a current cluster W has a subset S with $d(S) \leq \frac{1}{2}d(W)$ and $\Phi(S, S \subseteq W) \leq \epsilon$, then split W into two clusters S and W/S . Repeat until no such split is possible.

Where

$$\Phi(S, T) = \frac{\text{Number of edges from } S \text{ to } T}{\text{Total number of edges incident to } S \text{ in } G}.$$

Theorem

At termination of Recursive Clustering, the total number of edges between vertices in different clusters is at most $O(\epsilon m \ln n)$.

proof

Each edge between two different clusters at the end was deleted at some stage by the algorithm. We will "charge" edge deletes to vertices and bound the total charge. When the algorithm partitions a cluster W into S and W/S with $d(S) \leq \frac{1}{2}d(W)$, each $k \in S$ is charged $\frac{d(k)}{d(W)}$ times the number of edges being deleted. Since $\Psi(S, W/S) \leq \epsilon$, the charge added to each $k \in W$ is at most $\epsilon d(k)$.

A vertex is charged only when it is in the smaller part, $d(S) \leq d(W)/2$, of the cut. So between any two times it is charged, $d(W)$ is reduced by a factor of at least two and so a vertex can be charged at most $\log_2 m \leq O(\ln n)$ times, proving the theorem.

Dense Submatrices and Communities

Represent n data points in d -space by the rows of an $n \times d$ matrix A . Assume that A has all nonnegative entries. We address the question of how to define and find efficiently a coherent large subset of rows. To this end, the matrix A can be represented by a bipartite graph.

Dense Submatrices and Communities

We want a subset S of row vertices and a subset T of column vertices so that

$$A(S, T) = \sum_{i \in S, j \in T} a_{ij}$$

is high.

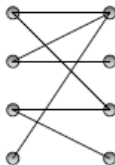
Remark

This definition is not good since $A(S, T)$ will be maximized by taking all rows and columns. Thus we define the density $d(S, T)$ of S and T , which is

$$d(S, T) = \frac{A(S, T)}{\sqrt{|S||T|}}.$$

Specially, A 's rows and columns both represent the same set and a_{ij} is the similarity between object i and object j . If A is an $n \times n$ 0 – 1 matrix, it can be thought of as the adjacent matrix of an undirected graph, and $d(s,s)$ is the average degree of a vertex in S . The subgraph of maximum average degree in a graph can be found exactly by network flow technique, this will show in next section.

A bipartite graph.png



Dense Submatrices and Communities

Theorem 7.10

Let A be an $n \times d$ matrix with entries between 0 and 1. Then

$$\sigma_1(A) \geq d(A) \geq \frac{\sigma_1(A)}{4 \log n \log d}.$$

Furthermore, subsets S and T satisfying $d(S, T) \geq \frac{\sigma_1(A)}{4 \log n \log d}$ may be found from the top singular vector of A .

Dense Submatrices and Communities

Proof

Let S and T be the subsets of rows and columns that achieve $d(A)=d(S,T)$. Consider an n -vector u that is $\frac{1}{\sqrt{|S|}}$ on S and 0 elsewhere and a d -vector v that is $\frac{1}{\sqrt{|T|}}$ on T and 0 elsewhere. Then,

$$\sigma_1(A) \geq u^T A v = \sigma_{i,j} u_i v_j a_{ij} = d(S, T) = d(A)$$

establishing the first inequality.

Dense Submatrices and Communities

To prove the second inequality, express $\sigma_1(A)$ in terms of the first left and right singular vectors x and y .

$$\sigma_1(A) = x^T A y = \sum_{i,j} x_i a_{ij} y_j, \quad |x| = |y| = 1.$$

Since the entries of A are nonnegative, the components of the first left and right singular vectors must all be nonnegative, that is, $x_i \geq 0$ and $y_j \geq 0$ for all i and j . To bound $\sum_{i,j} x_i a_{ij} y_j$, break the summation into $O(\log n \log d)$ parts. Each part corresponds to a given α and β and consists of all i such that $\alpha \leq x_i < 2\alpha$ and all j such that $\beta \leq y_j < 2\beta$. The $O(\log n \log d)$ parts are defined by breaking the rows into $\log n$ blocks with α equal to $\frac{1}{2} \frac{1}{\sqrt{n}}, \frac{1}{\sqrt{n}}, 2 \frac{1}{\sqrt{n}}, 4 \frac{1}{\sqrt{n}}, \dots, 1$ and by breaking the columns into $\log d$ blocks with β equal to $\frac{1}{2} \frac{1}{\sqrt{d}}, \frac{1}{\sqrt{d}}, 2 \frac{1}{\sqrt{d}}, 4 \frac{1}{\sqrt{d}}, \dots, 1$.

Dense Submatrices and Communities

The i such that $x_i < \frac{1}{2} \frac{1}{\sqrt{n}}$ and the j such that $y_j < \frac{1}{2} \frac{1}{\sqrt{d}}$ will be ignored at a loss of at most $\frac{1}{4} \sigma_1(A)$.

Since $\sum_i x_i^2 = 1$, the set $S = \{i \mid \alpha \leq x_i < 2\alpha\}$ has $|S| \leq \frac{1}{\alpha^2}$ and similarly, $T = \{j \mid \beta \leq y_j < 2\beta\}$ has $|T| \leq \frac{1}{\beta^2}$. Thus,

$$\begin{aligned} \sum_{i \mid \alpha \leq x_i < 2\alpha} \sum_{j \mid \beta \leq y_j < 2\beta} x_i a_{ij} y_j &\leq 4\alpha\beta A(S, T) \\ &\leq 4\alpha\beta d(S, T) \sqrt{|S||T|} \\ &\leq 4d(S, T) \leq 4d(A). \end{aligned}$$

Which implies

$$d(A) \geq \frac{\sigma_1(A)}{4 \log n \log d}.$$

Dense Submatrices and Communities

Lemma

$d(A)^2 \leq f(A) \leq d(A) \log n$. Also, $\sigma_1(A)^2 \geq f(A) \geq \frac{c\sigma_1(A)^2}{\log n}$.
 $f(A)$ can be found exactly using flow techniques in later chapter.

Community Finding and Graph Partition

Given two sets of nodes S and T , define

$$E(S, T) = \sum_{i \in S, j \in T} e_{ij}.$$

Then we define the density of a set S to be

$$d(S, S) = \frac{E(S, S)}{|S|}.$$

Remark

If G is an undirected graph, then $d(S, S)$ can be viewed as the average degree in the vertex-induced subgraph over S . The set S of maximum density is therefore the subgraph of maximum average degree.

Flow Method

- Here we consider dense induced subgraphs of a graph. Which means maximizing the density $d(S, S)$ over all subsets S of the graph.

Given a graph G in which one wants to find a dense subgraph, construct a directed graph H from the given graph and then carry out a flow computation on H .

H has a node for each edge of the original graph, a node for each vertex of the original graph, plus two additional nodes s and t . There is a directed edge with capacity one from s to each node corresponding to an edge of the original graph and a directed edge with infinite capacity from each node corresponding to an edge of the original graph to the two nodes corresponding to the vertices the edge connects.

Finally, there is a directed edge with capacity λ from each node corresponding to a vertex of the original graph to t .

Dense subgraph.png

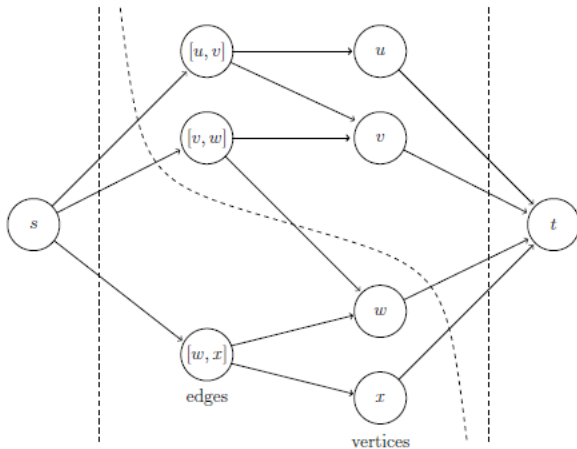


Figure 8.5: The directed graph H used by the flow technique to find a dense subgraph

Flow Method

Example

Consider finding a dense subgraph of 1,000 vertices and 2,000 internal edges in a graph of 10^6 vertices and 6×10^6 edges. For concreteness, assume the graph was generated by the following process. First, a 1,000-vertex graph with 2,000 edges was generated as a random regular degree four graph. The 1,000-vertex graph was then augmented to have 106 vertices and edges were added at random until all vertices were of degree 12. Note that each vertex among the first 1,000 has four edges to other vertices among the first 1,000 and eight edges to other vertices.

Flow Method

Example

The graph on the 1,000 vertices is much denser than the whole graph in some sense. Although the subgraph induced by the 1,000 vertices has four edges per vertex and the full graph has twelve edges per vertex, the probability of two vertices of the 1,000 being connected by an edge is much higher than for the graph as a whole. The probability is given by the ratio of the actual number of edges connecting vertices among the 1,000 to the number of possible edges if the vertices formed a complete graph.

$$p = \frac{e}{\binom{v}{2}} = \frac{2e}{v(v-1)}$$

Flow Method

Example

For the 1,000 vertices, this number is $p = \frac{2 \times 2000}{1000 \times 999} \cong 4 \times 10^{-3}$. For the entire graph this number is $p = \frac{2 \times 6 \times 2000}{10^6 \times 10^6} = 12 \times 10^{-6}$. This difference in probability of two vertices being connected should allow us to find the dense subgraph.

Cut in flow graph.png

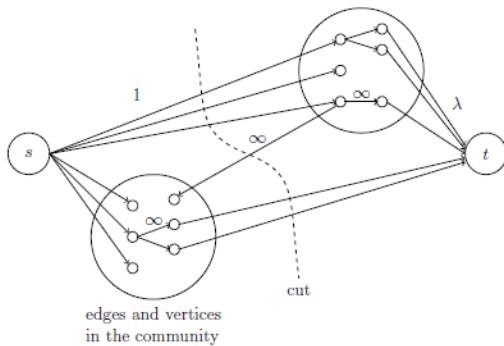


Figure 8.6: Cut in flow graph

Summary

- High-density Cluster: Single linkage and Robust linkage.
- Kernel Method: Center-based and density-based clustering.
- Recursive clustering: Based on Spares cuts .
- Dense Submatrices and Community: density of affinity matrix and Weighted graph

Thanks for your attention!